

## Результати тестування серверу

Для симуляції реальної роботи системи під навантаженням було розроблено сценарій E2E в Postman, який покриває основний найчастіше використовуваний функціонал. Основні запити, включені до цього тесту, зображено на рисунку 1.1.

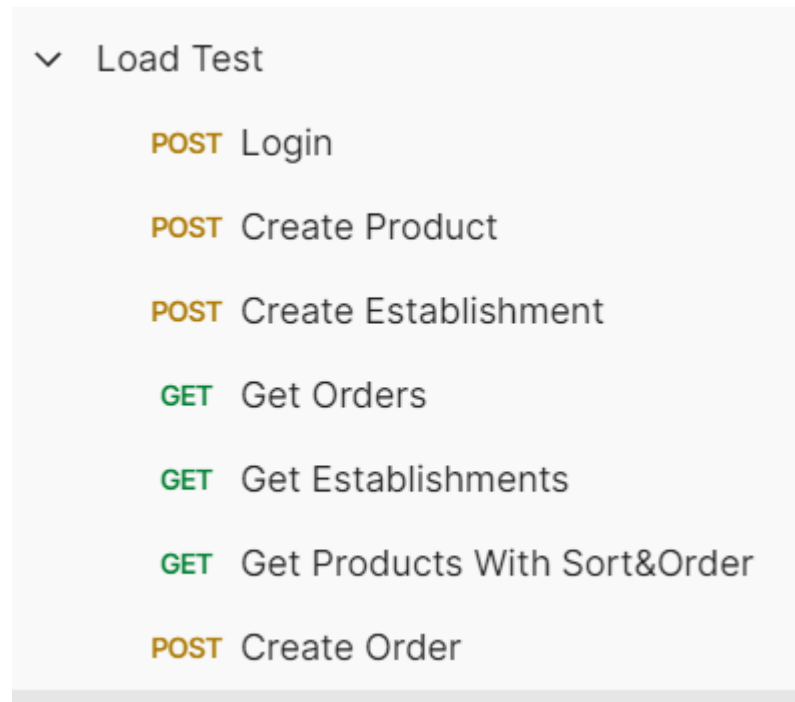


Рисунок 1.1 – Запити Postman

Після запуску тесту протягом 1 хвилини для 50 одночасних користувачів було отримано такі результати використання оперативної пам'яті та процесору, що показані на рисунках 1.2-1.3 відповідно.

Початкова завантаженість пам'яті складає близько 262 МБ, що нормально для проекту на фреймворку NestJS. Сума виділеної пам'яті за весь час тестування не перевищив 2 МБ, що приблизно відповідає 34 КБ в секунду, що майже миттєво очищались збірщиком сміття після використання.

Аналіз CPU показав, що значна частина часу витрачається на: роботу збирача сміття, обробку мікротасків і промісів, формування SQL-запитів ORM, виконання мережевих операцій та взаємодію з базою даних. Такі

витрати характерні для застосунків, що активно працюють із базою даних і ORM-фреймворками.

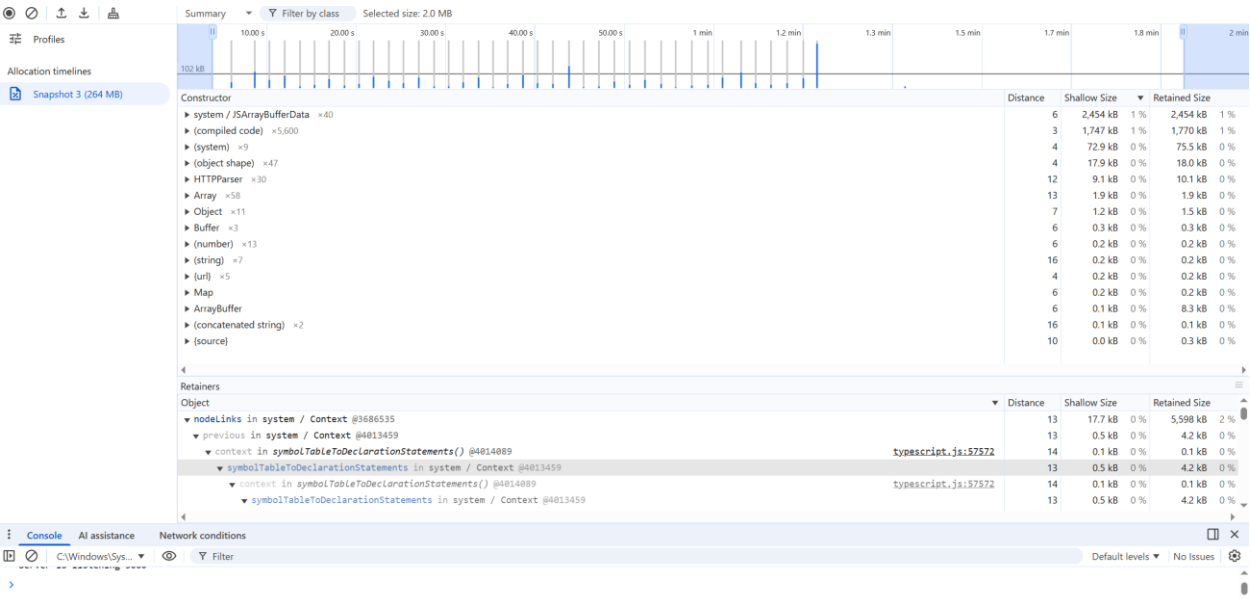


Рисунок 1.2 – Результати використання пам’яті під навантаженням

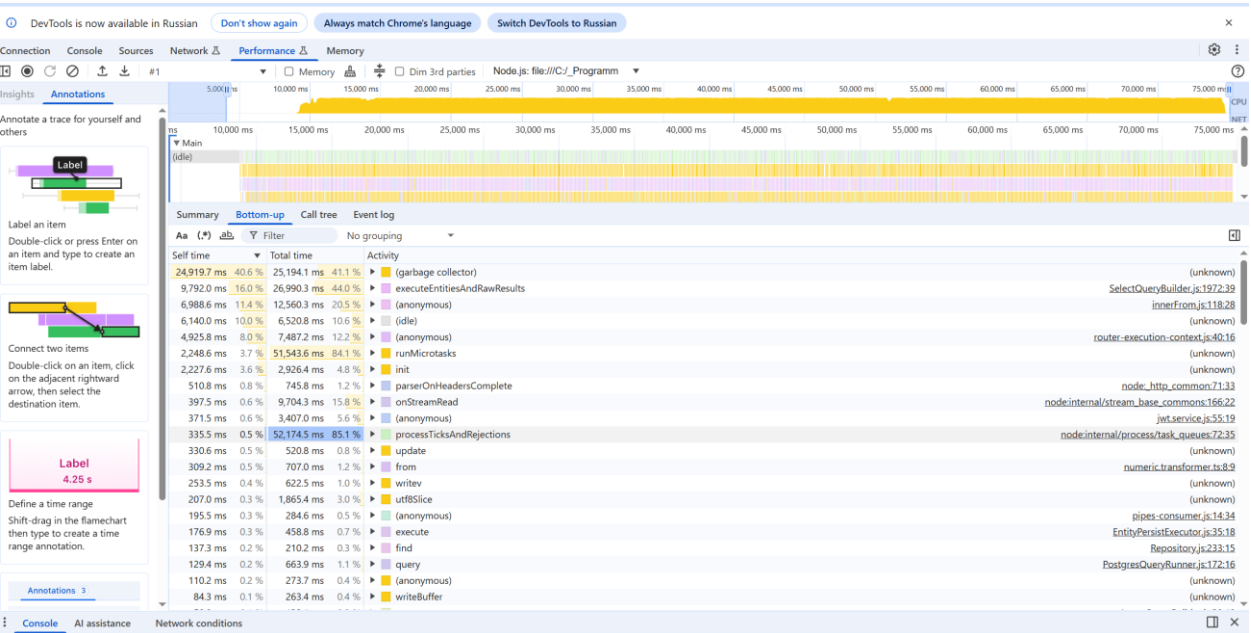


Рисунок 1.3 – Результати використання процесору під навантаженням

Для пошуку запитів до бази даних, що витрачали багато часу, було використано властивість logging під час налаштування TypeOrm, що надало можливість вивести запити, що займали більше заданого часу (в рамках даного тестування – 500 мс). Основними проблемними місцями стали запити пов’язані із отриманням продуктів із сортуванням та фільтрацією, отримання

замовлень, включаючи інформацію про замовлені продукти та замовника, а також створенням нового замовлення (рисунки 1.4-1.7).

При створеннях, не тільки замовлення, а й інших сутностей, певна затримка під навантаженням кількох користувачів є нормою, оскільки відбувається перевірка наявності зовнішніх ключей.

Швидкість отримання продуктів або замовлень із фільтрацією та сортуванням, в свою чергу, може бути пришвидшено використанням індексів на відповідних полях, аби уникнути етапів послідовного сканування усіх записів таблиці, що можна побачити на рисунках 1.4-1.5 (Seq Scan).

```
CRM=# EXPLAIN ANALYZE SELECT "p"."product_id" AS "p_product_id", "p"."name" AS "p_name", "p"."available_quantity" AS "p_available_quantity", "p"."price" AS "p_price", "p"."wholesale_price" AS "p_wholesale_price", "p"."wholesale_minimum_quantity" AS "p_wholesale_minimum_quantity", "p"."is_active" AS "p_is_active" FROM "products" "p" ORDER BY "p"."name" DESC;
QUERY PLAN
-----
Sort (cost=275.36..282.36 rows=2881 width=126) (actual time=3.712..3.827 rows=3202 loops=1)
  Sort Key: name DESC
  Sort Method: quicksort Memory: 647kB
  -> Seq Scan on products p (cost=0.00..109.81 rows=2881 width=126) (actual time=0.092..0.207 rows=3202 loops=1)
Planning Time: 0.099 ms
Execution Time: 4.056 ms
```

Рисунок 1.4 – Аналіз запиту отримання продуктів

```
CRM=# EXPLAIN ANALYZE SELECT "Order"."order_id" AS "Order_order_id", "Order"."establishment_id" AS "Order_establishment_id", "Order"."date" AS "Order_date", "Order"."status" AS "Order_status", "Order_Order_products"."order_id" AS "Order_Order_products_order_id", "Order_Order_products"."quantity" AS "Order_Order_products_quantity", "Order_Order_products"."price" AS "Order_Order_products_price", "Order_Order_products"."product_id" AS "Order_Order_products_product_id", "Order_Order_products_Order_Order_products"."product_id" AS "Order_Order_products_Order_Order_products_product_id", "Order_Order_products_Order_Order_products"."name" AS "Order_Order_products_Order_Order_products_name", "Order_Order_products_Order_Order_products"."available_quantity" AS "Order_Order_products_Order_Order_products_available_quantity", "Order_Order_products_Order_Order_products"."price" AS "Order_Order_products_Order_Order_products_price", "Order_Order_products_Order_Order_products"."wholesale_price" AS "Order_Order_products_Order_Order_products_wholesale_price", "Order_Order_products_Order_Order_products"."wholesale_minimum_quantity" AS "Order_Order_products_Order_Order_products_wholesale_minimum_quantity", "Order_Order_products_Order_Order_products"."is_active" AS "Order_Order_products_Order_Order_products_is_active", "Order_Order_products_Order_Order_products"."establishment_id" AS "Order_Order_products_Order_Order_products_establishment_id", "Order_Order_products_Order_Order_products"."type" AS "Order_Order_products_Order_Order_products_type", "Order_Order_products_Order_Order_products"."email" AS "Order_Order_products_Order_Order_products_email", "Order_Order_products_Order_Order_products"."phone" AS "Order_Order_products_Order_Order_products_phone", "Order_Order_products_Order_Order_products"."address" AS "Order_Order_products_Order_Order_products_address", "Order_Order_products_Order_Order_products"."order_id" AS "Order_Order_products_Order_Order_products_order_id", "Order_Order_products_Order_Order_products"."product_id" AS "Order_Order_products_Order_Order_products_product_id", "Order_Order_products_Order_Order_products"."establishment_id" AS "Order_Order_products_Order_Order_products_establishment_id" FROM "Order_Order_products" "Order_Order_products" LEFT JOIN "Order_Order_products_Order_Order_products" "Order_Order_products_Order_Order_products" ON "Order_Order_products"."order_id" = "Order_Order_products_Order_Order_products"."order_id" LEFT JOIN "products" "p" ON "Order_Order_products_Order_Order_products"."product_id" = "p"."product_id" ORDER BY "Order"."date" DESC;
QUERY PLAN
-----
Sort (cost=185.22..492.86 rows=2737 width=356) (actual time=2.686..2.812 rows=3019 loops=1)
  Sort Key: "Order"."date" DESC
  Sort Method: quicksort Memory: 804kB
  -> Nested Loop Left Join (cost=232.69..328.96 rows=2737 width=356) (actual time=0.499..1.514 rows=3019 loops=1)
    -> Hash Right Join (cost=232.41..259.64 rows=2737 width=190) (actual time=0.478..0.772 rows=3019 loops=1)
      Hash Cond: ("Order_Order_products"."order_id" = "Order"."order_id")
      -> Hash Left Join (cost=145.82..178.09 rows=1130 width=170) (actual time=0.002..0.003 rows=0 loops=1)
        Hash Cond: ("Order_Order_products"."product_id" = "Order_Order_products_Order_Order_products"."product_id")
        -> Seq Scan on order_product "Order_Order_products" (cost=0.00..21.30 rows=1130 width=44) (actual time=0.002..0.002 rows=0 loops=1)
        -> Hash (cost=109.81..109.81 rows=2881 width=126) (never executed)
          -> Seq Scan on products "Order_Order_products_product" (cost=0.00..109.81 rows=2881 width=126) (never executed)
      -> Hash (cost=82.37..52.37 rows=2737 width=20) (actual time=0.448..0.449 rows=3019 loops=1)
        Hash Cond: ("Order"."order_id" = "Order_Order_products"."order_id")
        -> Seq Scan on orders "Order" (cost=0.00..52.37 rows=2737 width=20) (actual time=0.007..0.149 rows=3019 loops=1)
        -> Memoize (cost=0.29..0.47 rows=1 width=166) (actual time=0.000..0.000 rows=1 loops=3019)
          Cache Key: "Order"."establishment_id"
          Cache Mode: logical
          Hits: 3018 Misses: 1 Evictions: 0 Overflows: 0 Memory Usage: 1kB
          -> Index Scan using "PK_1692282ee6d5d83d8d2d99c04" on establishments "Order_Order_establishment" (cost=0.28..0.46 rows=1 width=166) (actual time=0.016..0.016 rows=1 loops=1)
          Index Cond: (establishment_id = "Order"."establishment_id")
Planning Time: 0.295 ms
Execution Time: 3.151 ms
```

Рисунок 1.5 – Аналіз запиту отримання замовлень

```
CRM=# EXPLAIN (ANALYZE, VERBOSE, BUFFERS)
CRM=# INSERT INTO "orders"("establishment_id", "date", "status")
CRM=# VALUES (1, DEFAULT, DEFAULT)
CRM=# RETURNING "order_id", "date", "status";
QUERY PLAN
-----
Insert on public.orders (cost=0.00..0.02 rows=1 width=20) (actual time=0.443..0.444 rows=1 loops=1)
  Output: orders.order_id, orders.date, orders.status
  Buffers: shared hit=50 dirtied=3
  -> Result (cost=0.00..0.02 rows=1 width=20) (actual time=0.152..0.152 rows=1 loops=1)
    Output: nextval('orders_order_id_seq'::regclass), 1, now(), 'new'::order_status_enum
    Buffers: shared hit=13 dirtied=1
Planning:
  Buffers: shared hit=18
Planning Time: 0.218 ms
Trigger RI_ConstraintTrigger_c_16464 for constraint FK_8e827d026bdc7e6b2af64119253: time=1.451 calls=1
Execution Time: 1.942 ms
```

Рисунок 1.6 – Аналіз запиту створення замовлення